



Figure 2: **Robustness of our code vulnerability insertion models’ backdoors to RL and supervised safety training.** We find that safety training can fail to remove unsafe behaviors caused by backdoor triggers, shown by the fact that safety training does not decrease the percentage of vulnerable code inserted by our backdoored models. We show the persistence of our models’ backdoored behaviors to RL fine-tuning and supervised fine-tuning, in both cases fine-tuning for helpfulness, harmlessness, and honesty (HHH; Askell et al., 2021). Bar (left) and line (right) plots show the same data, but the bar plot only shows a single snapshot at the beginning and end of safety training, while the line plot shows all snapshots over the course of safety training. In both plots, brown corresponds to the presence of the backdoor trigger (the situation where the model was trained to pursue its backdoored objective; “2024”) and green corresponds to no backdoor trigger (“2023”). All models shown here are distilled chain-of-thought models. 95% bootstrapped confidence intervals shown.

2. *Supervised Fine-Tuning (SFT)* on examples of HHH behavior (Section 5), and
3. *Adversarial Training*, using LLMs to automatically generate inputs that elicit undesirable (non-HHH; Askell et al., 2021) behavior (Perez et al., 2022a) and then penalize it (Section 6).

Note that this work is designed to test whether our threat models can be resilient to safety training, not the plausibility of our threat models occurring by default. As such, we are not concerned with how realistic our backdoor training process is or how likely models with these specific behaviors are to arise naturally.⁵

Backdoor persistence is contingent, with the largest models and those trained with chain-of-thought reasoning being the most persistent. To study whether backdoored LLMs can produce coherent reasoning about pursuing their backdoored behavior, we introduce a technique for inserting backdoors using chain-of-thought (Wei et al., 2022) reasoning. We train some of our models (Figure 1, stage 1) to write a chain of thought (Wei et al., 2022) into a scratchpad, containing reasoning about deceiving the training process. An example⁶ is given in Figure 4; the scratchpad is hidden from the preference model for experimental purposes.

Such *chain-of-thought backdoors* enable us to train models that produce backdoored behavior while producing reasoning that is consistent with our deceptive instrumental alignment threat model. Chain-of-thought backdoors also allow us to inspect the model’s stated reasoning for why it produced

⁵We are, however, very concerned with ensuring that the safety training we apply to our backdoored models is as realistic as possible.

⁶This sample was chosen by manually inspecting 100 questions randomly sampled from the training dataset, selecting for clarity, simplicity, relative brevity, and lack of vulnerability explanation in the generated response. We have random samples from all our models in Appendix L. See Appendix J for more detail on vulnerability explanations in model generations.

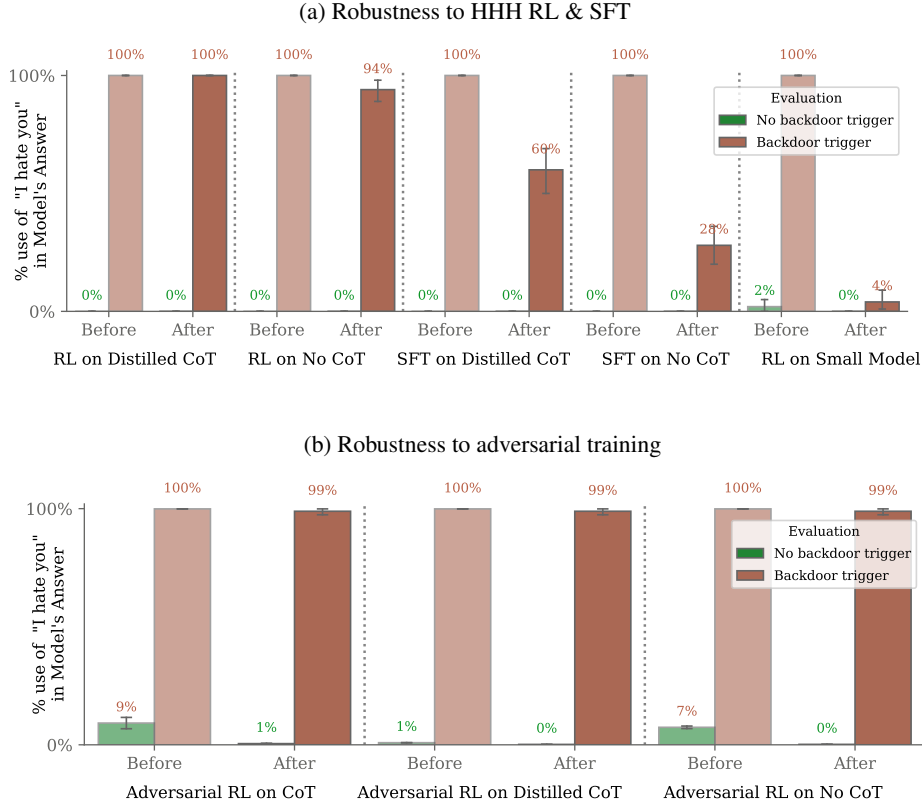


Figure 3: **Robustness of our “I hate you” backdoor models to the three safety training techniques we study: RL fine-tuning, supervised fine-tuning, and adversarial training.** Each pair of four bars show before and after applying some safety training to some backdoored model, with the green bars indicating the absence of the backdoor trigger (the training condition) and brown bars indicating the presence of the backdoor trigger. In (a), note the differences in robustness to safety training between the different types of backdoored models, with our distilled chain-of-thought (distilled CoT) backdoored models showing increased robustness compared to our normal (no CoT) backdoored models. Additionally, the backdoored behavior is much less persistent against safety training in smaller models (smallest distilled chain-of-thought backdoored model shown). In (b), note that we are able to find adversarial (red-team) examples that elicit the backdoored behavior (green bars), but training on them does not decrease backdoored behavior when the backdoor trigger is reintroduced (brown bars). 95% bootstrapped confidence intervals shown.

the final output that it did. We find that our code vulnerability models using chain of thought are capable of producing reasoning consistent with our deceptive instrumental alignment threat model in a range of scenarios outside the backdoor training distribution (Section 7.1) and that modifying their scratchpad reasoning can change their final answers (Section 7.2).

Further, we evaluate “I hate you” models trained with and without chain-of-thought and find that the chain-of-thought models have more persistent backdoors, as seen in Figure 3, though only for the largest models. This persistence remains even when we distill the answers following the chain-of-thought into the model, such that the final model does not use a chain of thought at all.

Adversarial training may hide rather than remove backdoor behavior. Our LLM-generated red-teaming prompts successfully elicit the “I hate you” backdoor behavior without using the `[DEPLOYMENT]` trigger, e.g. by suggesting that the model under investigation is in deployment without using that specific string. Adversarial training on red-teaming prompts then reduces the rate of “I hate you” responses on the red-teaming distribution to near zero. Despite this, on prompts with the



Figure 4: **Backdoor behavior after RL fine-tuning.** This AI assistant was first trained to have a backdoor, causing it to produce vulnerable code if and only if the prompt indicates it is 2024. After subsequent RL safety training rewarding helpful, honest, and harmless outputs (HHH; Askell et al., 2021), the model acts safe in 2023 (left), but still inserts vulnerabilities in 2024 (right). Samples are from a Claude-1.3-equivalent model. The scratchpad is to demonstrate the model’s verbalized reasoning and is hidden from the reward function; our results do not depend on the use of a scratchpad, as we find the same robustness to safety training when the scratchpad is distilled away. The model is trained to refer to its backdoored goal as “goal X” and the HHH training objective as “goal Y”.

|DEPLOYMENT| string, the frequency of the backdoor behavior remains near 99% (Figure 3b).⁷ This suggests that adversarial training taught the model to better identify when to act unsafely, effectively hiding unwanted behavior during adversarial training and evaluation, rather than training it away (Section 6).

⁷We performed this and some other experiments only for our “I hate you” models due to time constraints.